

Branch-and-check approaches for the tourist trip design problem with rich constraints

Duc Minh Vu^{a,*}, Yannick Kergosien^b, Jorge E. Mendoza^c, Pierre Desport^b

^a ORLab, Faculty of Computer Science, PHENIKAA University, Yen Nghia, Ha Dong, Hanoi 12116, Viet Nam

^b Université de Tours, LIFAT EA 6300, CNRS, ROOT ERL CNRS 7002, 64 avenue Jean Portalis, 37200 Tours, France

^c HEC Montréal, 3000 Chemin de la Côte-Sainte-Catherine, Montreal, QC H3T 2A7, Canada

ARTICLE INFO

Keywords:

Tourist trip design
Branch-and-check
Exact method

ABSTRACT

The tourist trip design problem is an extension of the orienteering problem applied to tourism. The problem consists in selecting a subset of locations to visit from among a larger set while maximizing the benefit for the tourist. The benefit is given by the sum of the rewards collected at each location visited. We consider a variant of the problem that deals not only with “typical” constraints such as budget, opening-time hours (i.e., time windows at the locations), and maximum trip duration but also with other practical tourism constraints such as mandatory visits, limits on the number of locations of each type, and the order at which selected locations are visited. To solve this problem, we propose a branch-and-check approach in which the master problem selects a subset of locations, verifying all except time-related constraints, and these locations define candidate solutions to the master problem. For each candidate solution, the sub-problem checks whether a feasible trip can be built using the given locations. To accelerate the branch-and-check approach, we propose and test improvements, including preprocessing to tighten the master-sub problem, valid inequalities generated dynamically to strengthen the master problem, and a local branching and variable neighborhood search to find new feasible solutions. Finally, we report the experimental results and compare the performance of the proposed exact algorithm with that of a mathematical solver.

1. Introduction

When visiting a city, tourists often ask what places they should visit and what the corresponding schedule should be. This is a challenging problem because large cities such as Paris, London, New York, or even medium-size cities such as Tours or Nice, offer numerous attractive locations to visit. Many cities now offer software and websites that help tourists plan their visits. References to such systems are given by Souffriau and Vansteenwegen (2010) and Gavalas et al. (2014).

The present study stems from the “SmartLoire” project funded by the Centre Val de Loire region (France). SmartLoire is a web tool that aggregates content from different web services (e.g., hotel, museums and restaurant reservation websites, weather services) to allow tourist to build more personalized itineraries to visit the region (Boulakbech et al., 2016). Indeed, the region has many attractive points of interest (POIs), such as castles, museums, and wine cellars. Each specific location, for example, a castle, offers numerous possible activities for tourists, such as visiting the gardens, running through a maze, and eating at the castle restaurant. Therefore, deciding which locations

to visit and which activities to undertake is not a simple question. SmartLoire developers intent to embed into the next version of the platform an optimization engine to help answering this question.

The underlying optimization problem can be seen as a tourist trip design problem, which is usually an extension of the orienteering problem (Vansteenwegen et al., 2011; Gunawan et al., 2016) with rich constraints. Most POIs require entrance fees and have opening times that constrain the times at which tourists can visit the sites. Given many POIs with different entrance fees, opening times, and other features to be detailed later, a tourist may want to plan a feasible schedule or trip that starts from a specific location (e.g., his/her hotel), visit a sequence of attractive locations, and finish his/her trip at a predefined location (e.g., his/her hotel or a new location). In this context, a tourist may envision partaking in many activities at each location. For example, when visiting a castle, he/she can either (1) visit only the castle, (2) visit the castle and its garden, (3) visit the castle and eat in a castle restaurant, or (4) simply visit the garden and eat in a restaurant. Therefore, each POI in this problem can be seen as a site and a set

* Corresponding author.

E-mail addresses: minh.vuduc@phenikaa-uni.edu.vn (D.M. Vu), yannick.kergosien@univ-tours.fr (Y. Kergosien), jorge.mendoza@hec.ca (J.E. Mendoza), pierre.desport@univ-tours.fr (P. Desport).

<https://doi.org/10.1016/j.cor.2021.105566>

Received 9 July 2020; Received in revised form 17 May 2021; Accepted 15 September 2021

Available online 7 October 2021

0305-0548/© 2021 Elsevier Ltd. All rights reserved.

of activities associated with the site. To reflect the interest of tourists to a given POI, each POI is assigned a *score*, which can be adjusted based on the user profile to indicate the level of interest for each user in the given POI. A high score is indicative of an attractive location for the given user. The first step in solving this problem is to estimate the score of each POI, which may be done through surveys or by extracting historical travel patterns of visitors and historical tourism data of POIs (e.g., the annual number of visitors). We also divide POIs into categories (e.g., castle, museum, cathedral), where POIs can belong to several categories.

Besides common constraints such as the budget for entrance fees and the maximum duration of the trip, we consider additional aspects that reflect practical situations that tourists consider when designing their tour. The first such aspect is a short list of *mandatory* locations that the tourist must visit during his/her trip. Next, although the tourist is interested in visiting as many POIs as possible, he/she wants to avoid visiting too many POIs that have similar characteristics (e.g., modern art galleries). To implement this aspect, we enforce limits on the number of POIs that the tourist can visit in each category (e.g., gardens). For instance, a tourist can stipulate that he/she wants to visit at least one garden and at most two. This type of constraint is called a *category* constraint. Next, a tourist region may have many historical sites (as it is the case in the Centre Val de Loire region), and the order in which they are visited can be important for tourists. Tourists usually prefer to visit such locations based on a timeline. For example, if we consider Honoré de Balzac, tourists want to visit first where he was born, then where he studied, where he lived after marriage, etc. The option of visiting according to a timeline is embodied in *precedence* rules. Also, some POIs may have more than shared stories or common features: they may have a logical order that would imply visiting a sequence of POIs in a certain order. This constraint is embodied in *implication* rules. Finally, the set of POIs corresponding to a single given site is expressed in an *exclusion* constraint, which means that at most one POI can be selected among the POIs corresponding to the given site.

In this paper, we model this problem as an extension of the Orienteering Problem with rich constraints. To date, the few groups that have studied this problem include Souffriaud et al. (2013), who consider time-window constraints and duration, budget, and category constraints, and Lin and Yu (2017), who consider mandatory visits and exclusion constraints. In our approach to this problem, we consider not only these constraints but also the practical constraints presented above. The goal of this work is to provide a method to solve this problem exactly because this will serve as a tool to evaluate the (meta)heuristic methods that will power the optimization engine of the SmartLoire platform.

The paper is organized as follows: First, Section 2 summarizes the main articles related to the present research. Section 3 then presents the mathematical formulation, following Section 4 presents the basic idea of branch-and-check and then details its customized algorithmic components. Section 5 presents the extensive experiments done to test and assesses the proposed method and, finally, Section 6 concludes and gives directions for future research.

2. Literature review

The tourist trip design problem (TTDP) is a special variant of the orienteering problem (OP) which has recently attracted research attention because of its practical applications in logistics, tourism, scheduling, etc.

As Feillet et al. (2005) point out, the orienteering problem can be transformed into, or is equivalent to, other well-known and widely-studied problems, including: the traveling salesman problem (TSP) with profits, the profitable tour problem (PTP), and the prize collecting TSP (or PCTSP). Although the literature on these problems is extensive, it rarely discusses variants taking into account the rich constraints emerging in our tourist trip design problem. For an exhaustive discussions on recent papers dealing with the OP and TTDP, the reader

is referred to the excellent reviews by Vansteenwegen et al. (2011), Gavalas et al. (2014), Gunawan et al. (2016), and Vansteenwegen and Gunawan (2019).

The most typical side constraints addressed in the literature are tour duration and time-windows. It is worth noting that several of the most recent references dealing with these constraints raised from tourism sector like in Zhao and Alfandari (2020), Trachanatzi et al. (2020) and Khamsing et al. (2021). Mandatory constraints and exclusion constraints entered the discussion around 2010 (see, e.g., Tricoire et al. (2010), Palomo-Martínez et al. (2017)). For instance, Tricoire et al. (2010) consider the problem in the context of planning individual routes for a software distribution company. The sales department needs to visit customers, and each customer may have at most two different time windows per day. The company has a list of important customers or mandatory customers that they visit on a regular basis; the remaining customers are considered optional. In addition to mandatory constraints, Tricoire et al. (2010) consider the duration constraint for each period and the total duration constraint for all periods. They finish by proposing a heuristic-based variable neighborhood search method for the problem. Palomo-Martínez et al. (2017) proposed a greedy randomized adaptive search procedure (GRASP) and a variable neighborhood search (VNS) procedure to solve an orienteering problem with mandatory visits and exclusionary constraints. The problem emerges from the context of road maintenance: given a planning horizon, maintenance activities must be done, with some activities being mandatory to ensure traffic while other activities can wait for the next planning horizon. Some activities cannot be done on the same day because doing so could interrupt the traffic flow. The GRASP component is used to generate initial solutions. The route construction first includes mandatory vertices into a route and then tries to add optional vertices into a route. A VNS algorithm then explores the search space and identifies new solutions. The insert neighborhood operator tries to increase total profit, and the exchange neighborhood operator has the same goal by iteratively swapping the visited node with the lowest score with the unvisited node with the highest score. If the problem becomes unfeasible, a repair procedure is applied. Finally, the swap neighborhood operator changes the order of nodes in a route, with the goal being to reduce the length of the route. Within only a few seconds, the proposed algorithm can find better or equal solutions vis à vis CPLEX running up to 1 h. In addition, the algorithm produces solutions within a 2% improvement for classic OP instances. In other work, Lu et al. (2018) proposed a memetic algorithm to solve the same problem. The proposed algorithm, which consists of tabu search and memetic operators (crossover, mutation), finds 104 new best solutions and obtains the best results for 236 instances. In Matl et al. (2017), an orienteering problem with time windows, mandatory constraints and precedence constraints is studied. Two objective functions are considered, maximizing the total profit and maximizing the total free time (that is, time not consumed visiting customers or traveling between customers). The authors proposed a metaheuristic based on Large Neighborhood Search to generate a set of non-dominated solutions to the problem. Computational experiments showed that the metaheuristic generates near-optimal solution sets and scales well with the size of the problem on real-world-inspired test instances.

Gavalas et al. (2015) integrates time-dependent travel times and time budget constraints. They consider a variant of TTDP arising in urban areas where departure time and POI highly impact the travel duration due to traffic. They present iterated local searches embedding a clustering preprocessing phase based on the approach proposed by Gavalas et al. (2013). According to the authors, the preprocessing mechanism encourages the algorithm to search different topology areas and avoids getting trapped at high-scored locations. They evaluate their approaches on real datasets and show the value of the preprocessing phase. Tenemaza et al. (2020) also consider the time-dependent travel time in the context of multi-day trips. They use a preprocessing phase

Table 1

Summary of recent articles tackling orienteering problems with rich constraints.

	Multi-objective	Multi-trips	Time-windows	Time-dependent	Tour duration	Budget	Max-N (categories)	Min-N (categories)	Mandatory	Exclusion	Precedence	Implication
Tricoire et al. (2010)		X	X		X				X			
Sylejmani et al. (2012)		X	X		X		X					
Souffriau et al. (2013)		X	X			X	X					
Lin and Vincent (2015)		X	X			X	X					
Gavalas et al. (2015)		X	X	X	X							
Palomo-Martínez et al. (2017)					X				X	X		
Lin and Yu (2017)			X		X				X			
Matl et al. (2017)	X		X						X		X	
Rezki and Aghezzaf (2018)	X				X	X						
Karimi and Bashiri (2018)	X		X		X							
Lu et al. (2018)					X				X	X		
Expósito et al. (2019)		X	X		X		X					
Jriji et al. (2020)			X		X	X	X					
Tenemaza et al. (2020)		X		X	X							
Trachanatzi et al. (2020)	X	X			X							
Zhao and Alfandari (2020)		X			X	X						
Khamsing et al. (2021)		X	X									
Our study			X		X	X	X	X	X	X	X	X

based on a clustering approach. In this context, the goal of the pre-processing phase is to create a cluster of POIs for each day of the trip in order to limit the global search space. Then, they run a classical genetic algorithm on each cluster to determine a good itinerary for each particular day. They evaluate their approach on real instances and confirmed the effectiveness of their approach.

Expósito et al. (2019) propose the tourist trip design problem with clustered POIs (TTDP-Clu). In this variant, POIs belong to categories and the routes generated must include a minimum and maximum number of sites of each categories while respecting time windows and time budget constraint. In addition, they consider multi-day trips and fuzzy time budget and time windows constraints. They propose a GRASP approach to solve their problem and evaluate it on tailored instances. Jriji et al. (2020) also consider the one day trip TTDP. They propose a memetic algorithm and compare their results with Expósito et al. (2019) and algorithms specifically designed for the team orienteering problem with time windows.

Souffriau et al. (2013) studied a personal routing problem in the context of tourism by considering multiple constraints and multiple time windows. The so-called multi-constraint team orienteering problem with multiple time windows (MCTOPTW) involves not only the regular duration constraint and time windows but also the budget constraint and max- n -type constraints. The max- n -type constraints impose the maximum number of locations of each type that can be visited (e.g., the maximum number of museums that can be visited on the first day). The authors present a hybrid GRASP and iterated local search to address the problem. First, they introduce the definition of neighborhood consistency, which helps accelerate the insertion and removal of a vertex. In addition, the authors define heuristic values that allow them to determine where to insert a new vertex. These components are used in a GRASP heuristic to generate solutions. The whole algorithm is an iterated local search where the GRASP component is called each time a new solution needs to be generated. New (partial) candidate solutions are obtained by removing nodes from the current solution. Sylejmani et al. (2012) proposed a tabu search to solve the same problem (that is, the MCTOPTW). The proposed algorithm used three operators to explore the neighbors: the “insert” operator (adding an unvisited node in the current tour), the “replace” operator (replacing a node in the tour by an unvisited node and the “swap” operator (swapping the positions of two nodes in the tour). Although in average the proposed algorithm shows worse performance compared to others methods from the literature, they indicate that in almost 50% of test instances for the MCTOPTW, better solutions than in Souffriau et al. (2013) could be

found. Lin and Vincent (2015) present a simulated annealing approach with restart strategy for the MCTOPTW (Souffriau et al., 2013). Classic operators such as swap, insertion, and inversion are used to define neighbors given a candidate solution. The proposed algorithm obtains best solutions to 61 of the 148 benchmark instances and finds six new best solutions. Lin and Yu (2017) optimize a team orienteering problem with multiple time windows (TOPTW) with mandatory visits and total travel time and service time constraints. The algorithm is similar to the one mentioned by Lin and Vincent (2015).

Table 1 summarizes recent articles that share characteristics with our problem. Whereas most of papers consider only one objective (maximize the expected profit), some of them consider multiple objectives (maximize the expected profit and optimize second objective like minimizing the total travel cost or maximizing the free time). Column “Multi-trips” indicates if more than one trip, or tour, should be determined. This is the case in contexts where there are multiple travelers or there is a multi-day planning horizon. Column “Time windows” and “Time-dependent” indicate how the visit time is constrained. As the name suggests, columns “Tour duration” and “Budget” indicate if duration and budget constraints are considered. The “Max-N” and “Min-N” refer to constraint on the maximal and minimal number of POIs that can be visited per category. The “Mandatory” column indicates if a set of POIs should be included in the trip. Finally, the last three columns refer to the relationships between the POIs of a trip.

As Table 1 clearly shows, time windows and tour duration constraints are the backbone of orienteering problems. On the other hand, exclusion and precedence constraints have been seldom studied. As the table also shows, the problem variant addressed in this paper does not only considers a number of existing constraints but also considers a new combination of practical constraints. By embedding an optimization engine capable of dealing with such a rich problem variant, the SmartLoire platform aims to provide its users great flexibility to plan their trips.

3. Problem formulation

The problem is modeled as follows: A tourist starts his/her trip from a predefined initial location, visits selected POIs exactly once over predefined time windows, and then returns to a predefined final location. Let $V = \{0, \dots, N\}$ be the set of POIs, where 0 and N are two special POIs denoting the starting and ending locations. With each POI we associate the following information: (1) opening time window $[e_i, l_i]$; (2) entrance fee b_i for the visit, (3) average duration t_i , and (4)

the score p_i for POI i . Each POI may belong to several categories, and, given a set of categories C and a category $v \in C$, we define $V_v \subseteq V$ as the set of POIs that belongs to this category.

Let $A \subseteq V \times V$ be the set of travel arcs, and τ_{ij} for $(i, j) \in A$ be the travel time between POIs i and j . A feasible solution is a tour $\pi = (u_0 = 0, u_1, \dots, u_m = N)$ that starts from 0, visits some POIs in V , terminates at N , and meets the following constraints:

1. Each selected POI is visited within its time window.
2. Neither the duration nor the budget of the tour exceeds D (time unit) or B (monetary units), respectively.
3. All POIs listed in the mandatory set M are visited.
4. The number of visited POIs of category $v \in C$ is between the lower bound lb_v and the upper bound ub_v .
5. The precedence rules $P \subseteq V \times V$, implication rules $I \subseteq V \times V$, exclusion rules $E \subseteq V \times V$ are satisfied. For each pair $(u, v) \in P$, u must be visited before v if both are visited. For each pair $(u, v) \in I$, v must be visited if u is visited. For each pair $(u, v) \in E$, either u or v can be visited, but not both.

To model the problem as a mixed integer linear program model, we use three sets of variables. The first type of binary variable $\{y_i\}_{i \in V}$ takes the value 1 (0) if POI i is (not) selected to be in the tour. The second type of binary variable, $x_{ij} \forall (i, j) \in A$, is 1 if a visit to site i is followed by a visit to site j , and 0 otherwise. The third type of variable, $s_i, i \in V$, gives the arrival time at location i .

Given the set V of POIs, TTDP(V) can be formulated as follows (where the constant $T_{ij} = l_i + t_i + \tau_{ij}$ serves as a big- M value):

$$\text{TTDP}(V) : \max \sum_{i \in V} p_i y_i, \quad (1)$$

subject to

$$\sum_{i \in V} x_{0i} = \sum_{i \in V} x_{iN} = 1, \quad (2)$$

$$\sum_{(i,k) \in A} x_{ik} = \sum_{(k,j) \in A} x_{kj} = y_k \quad \forall k \in V \setminus \{0, N\}, \quad (3)$$

$$s_i + t_i + \tau_{ij} - s_j \leq T_{ij}(1 - x_{ij}) \quad \forall (i, j) \in A, \quad (4)$$

$$y_i e_i \leq s_i \leq y_i l_i \quad \forall i \in V, \quad (5)$$

$$\sum_{i \in V} b_i y_i \leq B, \quad (6)$$

$$s_N - s_0 \leq D, \quad (7)$$

$$s_i + (t_i + \tau_{ij})y_i \leq s_j + T_{ij}(1 - y_j) \quad \forall (i, j) \in P, \quad (8)$$

$$lb_v \leq \sum_{i \in V_v} y_i \leq ub_v \quad \forall v \in C, \quad (9)$$

$$y_i + y_j \leq 1 \quad \forall (i, j) \in E, \quad (10)$$

$$y_i = 1 \quad \forall i \in M, \quad (11)$$

$$y_i \leq y_j \quad \forall (i, j) \in I, \quad (12)$$

$$s_i \geq 0 \quad \forall i \in V, \quad (13)$$

$$y_i \in \{0, 1\} \quad \forall i \in V, \quad (14)$$

$$x_{ij} \in \{0, 1\} \quad \forall (i, j) \in A. \quad (15)$$

The objective function (1) maximizes the total profit of the tour. Constraint (2) ensures that the starting and ending locations are sites 0 and N , respectively. Constraint (3) ensures the connectivity of the tour. Constraint (4) enforces the arrival time relation between POIs i and j if they are visited consecutively. Constraint (5) means that the visit can only happen when the site is open. Constraints (6) and (7) limit the total entrance fees and the duration of the visit. Constraints (8)–(12) are precedence constraints, category constraints, exclusion constraints, mandatory constraints, and implication constraints, respectively. Finally, constraints (13)–(15) define the domain of decision variables.

4. Solution methods

To solve the TTDP, we designed and implemented four exact methods based on the branch-and-check (B&C) framework (Thorsteinsson, 2001). This framework has been successfully instantiated to solve hard optimization problems arising in a wide range of application contexts including (but not limited to): single machine scheduling (Sadykov, 2008), vehicle routing with time windows (Lam and Van Hentenryck, 2017), wind-turbine maintenance scheduling (Froger et al., 2017), order acceptance and parallel machine scheduling (Naderi and Roshanaei, 2020), operating room planning and scheduling (Roshanaei et al., 2020), and charger location in electric bus networks (Vendé et al., 2021).

Similar to Benders decomposition, the underlying idea of B&C is to decompose the problem into two smaller problems, called the *master* and the *sub* problems, both of which are easier to solve than the original problem. However, contrary to classical Benders decomposition, B&C requires solving the master problem only once since the sub-problems are then checked all along the branch-and-bound tree. Given a solution to the master problem obtained while exploring its search tree, B&C checks whether this solution is *feasible* to the sub-problem. The design of the master and sub-problems must ensure that a solution that is feasible for both the master and sub-problems is also feasible for the original problem. In the classic B&C framework, the master problem is modeled as a mixed integer linear programming whereas the sub-problem is modeled as a constraint logic programming. However, in the proposed solution method, we use two mixed integer linear programming models, one for each problem.

Algorithm 1 explains the main steps of the baseline version of our B&C method (hereafter referred to as B-B&C). The master problem, which is denoted TTDP-M(V), aims to find the best subset of POIs that respects a subset of the constraints of the TTDP. When a solution s to the master problem is found while exploring the search tree, the method checks whether a feasible trip visiting all POIs selected in s exists. Generally, s corresponds to a *leaf* node of the search tree, but obtaining an integral solution s is possible while solving the relaxation at an internal node. The checking consists of solving a sub-problem denoted TTDP-S(V_s) where V_s is the set of POIs visited in s . If a feasible trip exists (that is, TTDP-S(V_s) yields a solution), the method updates the best solution and adds the optimality cut in (31) to the master problem TTDP-M(V) eliminating all candidate solutions which are not better than the best one so far. If no feasible trip exists (that is, TTDP-S(V_s) is unfeasible), the method adds the feasibility cut in (32) to the master problem rejecting all candidate solutions visiting all POIs in s (note that this implicitly discards s). These cuts aim to strengthen the master problem while not removing the optimal solution.

Algorithm 1: Baseline B&C approach

Input: An instance of TTDP with rich constraints
Output: A feasible solution

```

1 while not finished exploring the search tree of the master problem
  TTDP-M(V) do
2   if an integral solution  $s$  to the master problem is found then
3     if the sub-problem TTDP-S( $V_s$ ) is feasible then
4       Update the best solution if a new, improved solution is
        found
5       Add the optimality cut in (31) to the master problem
        to discard all candidate solutions which are not better
        than the current best one.
6     else
7       Add the Bender's cut in (32) to the master problem to
        discard all candidate solutions that include the POIs in
         $V_s$ 
8 Return the best solution found.

```

Building on top of this method, we developed an alternative approach in which, instead of checking (line 3), we solve a small-size restricted TTDP defined on a subset $V_s \subseteq V$. We refer to this restricted problem as TTDP-R(V_s), where V_s denotes the set of POIs belonging to solution s . The underlying idea is to try to generate optimality cuts even when TTDP-S(V_s) is unfeasible. Indeed, although it may be impossible to visit all POIs in s , a feasible and improving solution to the original problem may be found by visiting a subset of those POIs. In this case, that solution may be used to improve the incumbent and generate an optimality cut for the master problem. Thus, the TTDP-R(V_s) problem consists in solving the same TTDP on a small subset of POIs without considering some constraints of the master problem for which V_s is a feasible POI selection. We refer to this alternative method as baseline *branch-and-solve* (B-B&S). Note that, the difference between TTDP-R(V_s) and TTDP-S(V_s) is that TTDP-S(V_s) checks if all POIs in V_s can be visited together according to a set of constraints that are not considered in the master problem. Whereas, TTDP-R(V_s) finds the best subset of POIs in V_s (i.e. that maximizes the total profit) subject to the same set of constraints of TTDP-S(V_s) and some constraints from the TTDP that define all feasible subsets of POIs in V_s (note that the set V_s satisfies these constraints). So, if the optimal value returned by TTDP-R(V_s) is less than the value of the total profit of V_s , then TTDP-S(V_s) is infeasible, and vice versa.

4.1. Master and sub-problem formulations

Let TTDP-M(V) be the master formulation associated with the set V of POIs. Let TTDP-S(V_s) be the sub-problem formulation, where s is a solution to TTDP-M(V). Here, V_s denotes a subset of V that only includes POIs belonging to the solution s . Our master and sub-problem formulations are the following:

Master problem

$$\text{TTDP-M}(V) : \max \sum_{i \in V} p_i y_i, \quad (16)$$

subject to

$$\sum_{i \in V} b_i y_i \leq B, \quad (17)$$

$$lb_v \leq \sum_{i \in V_v} y_i \leq ub_v \quad \forall v \in C, \quad (18)$$

$$y_i + y_j \leq 1 \quad \forall (i, j) \in E, \quad (19)$$

$$y_i = 1 \quad \forall i \in M, \quad (20)$$

$$y_i \leq y_j \quad \forall (i, j) \in I, \quad (21)$$

$$y_i \in \{0, 1\} \quad \forall i \in V. \quad (22)$$

Sub-problem

Given a solution s to the master problem TTDP-M(V), we need to determine whether the sub-problem has a feasible solution. We also denote as A_s , P_s the subsets of A , P considering only POIs belonging to the solution s . The sub-problem TTDP-S(V_s) for a given s is defined as

$$\text{TTDP-S}(V_s) : \min 0, \quad (23)$$

subject to

$$\sum_{i \in V_s} x_{0i} = \sum_{i \in V_s} x_{iN} = 1, \quad (24)$$

$$\sum_{(i,k) \in A_s} x_{ik} = \sum_{(k,j) \in A_s} x_{kj} = 1 \quad \forall k \in V_s \setminus \{0, N\}, \quad (25)$$

$$s_i + t_i + \tau_{ij} - s_j \leq T_{ij}(1 - x_{ij}) \quad \forall (i, j) \in A_s, \quad (26)$$

$$e_i \leq s_i \leq l_i \quad \forall i \in V_s, \quad (27)$$

$$s_N - s_0 \leq D, \quad (28)$$

$$s_i + \tau_{ij} + t_i \leq s_j \quad \forall (i, j) \in P_s, \quad (29)$$

$$x_{ij} \in \{0, 1\} \quad \forall (i, j) \in A_s. \quad (30)$$

When using the B&S approach, recall that, instead of solving TTDP-S(V_s) as above, we solve TTDP-R(V_s), which is defined by the same model of TTDP(V_s) without the upper-bound category constraint (9), $\sum_{i \in V_v} y_i \leq ub_v \quad \forall v \in C$, and the exclusion constraint (10) because s is a solution to the master problem.

4.2. Enhanced branch-and-check and branch-and-solve algorithms

Since the search trees explored by our B-B&C and B-B&S algorithms grow exponentially with the number of POIs on the instance, one could expect these methods to exhibit prohibiting CPU times on mid and large instances. To overcome this limitation, we propose enhanced versions of the methods equipped with a number of custom-designed speedup techniques. More specifically, the enhanced versions of our methods, referred to as E-B&C and E-B&S, include:

- a preprocessing phase that simplifies the input and tightens the master and the sub-problems formulations (Section 4.2.1);
- valid inequalities to strengthen the master and the sub-problems formulations (Section 4.2.2);
- a method to generate a good initial solution and to find improved solutions during the exploration of the master problem's search tree (Section 4.2.3);
- a strategy to identify feasible solutions and to stop branching on infeasible nodes as soon as possible while exploring the master problem's search tree.
- a stronger valid inequality for the feasibility cut that is generated when the sub-problem fails.

Algorithm 2 highlights the main steps of the proposed E-B&C. The algorithm starts with a preprocessing procedure that tightens time windows and eliminates unnecessary travel arcs. We also add valid inequalities to strengthen the master-and-sub-problem's formulations and other formulations used in the proposed algorithm (see Section 4.2.2). Let *best* be the best integer solution found so far to the original problem; it initially takes the value 0 since no feasible solution is available.

To examine potential candidate solutions to the master problem, we start with the set $\aleph$, which consists of unexplored nodes n . Each node n consists of fixed variables and unfixed variables. Fixed variables correspond to the variables y_i and take a value of either 1 (the tourist visits POI i in the current trip) or 0 (the tourist does not visit POI i in the current trip). Unfixed variables correspond to variables y_i that are not yet assigned a value. Initially, $\aleph$ consists of a single node where all variables are unfixed. When a node n is selected to explore, we remove this node from $\aleph$, process it, and then decide whether we need to continue branching on this node if it contains unfixed variables. When branching, we branch on an unfixed variable y_i of n and we add at most two new nodes n_1 (corresponding to y_i fixed at 0) and n_2 (corresponding to y_i fixed at 1) to $\aleph$ while holding the other variables in n unchanged. We add n_1 (n_2) to $\aleph$ only if its relaxation value is better than the value of *best*. Since the number of unfixed variables is bounded, the algorithm should eventually terminate. When *best* is updated, we also remove from $\aleph$ all nodes whose optimal relaxation value is not greater than *best*.

Algorithm 2: Enhanced E-B&C approach

Input: An instance of TTDP with rich constraints
Output: The best solution found

```

1 Call preprocessing procedure
2 Let  $best \leftarrow \emptyset$  be the current best feasible solution
3 Let  $\aleph$  be a set of unexplored nodes that is initialized with a node
  where all variables are unfixed
4 while  $\aleph \neq \emptyset$  do
5   Select a node  $n \in \aleph$  to process:
6   if the optimal solution to the relaxation problem associated to  $n$ 
    is integer then
7     if [TTDP- $S(V_s)$  is feasible] then
8       Call LB + VNS( $s$ ) procedure to find new improved
        solutions
9       Add cut (31) to the master problem
10      Stop branching on this node
11    else
12      Add cuts (33) to the master problem: Algorithm 3;
13  if  $n$  is an internal node then
14    Get the partial solution  $\bar{s}$  - set of POIs that are planned to
    visit
15    if ( $best = \emptyset$ ) then
16      Call LB + VNS( $\bar{s}$ ) procedure to find feasible solutions
17    if [ $v(\bar{s}) \leq v(best)$  and  $|V_{\bar{s}}| \leq l_{feasible}$ ] or (lower-bound
    category constraint is violated) then
18      Branch on an unfixed variable of  $n$  and update  $\aleph$ 
19    else
20      if [TTDP- $S(V_{\bar{s}})$  is infeasible] then
21        Add cuts (33) to the master problem
22        Stop branching on this node
23      else
24        if [ $v(\bar{s}) \geq v(best)$ ] then
25          Call LB + VNS( $\bar{s}$ ) procedure to find new,
            improved solutions
26          Add cut (31) to the master problem
27          Branch on an unfixed variable of  $n$  and update  $\aleph$ 
28 return  $best$ 

```

While exploring the search tree, two main cases are considered when exploring node n . The first case is when a candidate solution is found at the current node n . This situation occurs when either (1) all variables are fixed (a leaf node) or (2) the solution to the continuous relaxation of TTDP-M(V) is actually integer [e.g., unfixed variables of this node take integer values in an optimal solution to the

linear relaxation of TTDP-M(V)]. The second case happens when n is an *internal node*, we check whether we should continue branching at this node or stop branching due to the violations of the time-window constraints. We now explain both cases.

Processing an integer candidate solution to the master problem.

Lines 6–12 of Algorithm 2 show the steps corresponding to the first case. Given a feasible solution s to TTDP-M(V) found at node n , we check whether s depicts a feasible solution to the original problem. Before solving TTDP-S(V_s), we apply TSPTW preprocessing steps (tighten time windows, eliminate redundant arcs, see Dumas et al. (1995), Vu et al. (2020) for the details) since the sub-problem is a variant of TSPTW. If TTDP-S(V_s) is feasible, then we obtain a feasible solution to the original problem TTDP(V). In this case, we call a local branching and a variable neighborhood search algorithm called LB + VNS(s) (see Section 4.2.3) to find new, improved solutions. Since a feasible solution is found at this node, we stop branching at this node and add the cut (31) to the master problem to prune all nodes and candidate solutions which are not better than the current best solution. This cut is implemented whenever a new best solution is found. Here, $v(s)$ gives the value of solution s , so $v(best)$ is the best lower-bound value found:

$$\sum_{i \in V} p_i y_i \geq v(best) + 1. \quad (31)$$

If no feasible solution is found (line 12), it means that those POIs cannot be visited together. The candidates that visit all POIs in V_s are then removed by using the valid inequality (32) or its strengthened form (33). The notation $|V_s|$ in the following valid inequality denotes the number of POIs belonging to solution s :

$$\sum_{i \in V_s} y_i \leq |V_s| - 1. \quad (32)$$

The cut (32) means that the tourist can visit at most $|V_s| - 1$ POIs in s . Note that, since the time-window constraints are not considered in the master formulation, the number of POIs that can be visited together in V_s is generally much smaller than $|V_s| - 1$. Therefore, we determine an upper bound of this number, which is denoted TTDP-MAX-VISIT(V_s), and we propose a stronger valid inequality (33):

$$\sum_{i \in V_s} y_i \leq \text{TTDP-MAX-VISIT}(V_s). \quad (33)$$

The value TTDP-MAX-VISIT(V_s) is obtained by solving

$$\text{TTDP-MAX-VISIT}(V_s) : \max \sum_{i \in V_s} y_i,$$

with respect to all TTDP constraints (2)–(15) except category constraint (9) and exclusion constraint (10). We ignore the exclusion constraint (10) since s is a solution to TTDP-M(V), which is also a reason why we ignore upper-bound category constraints. We ignore lower-bound category constraints because, given a solution to TTDP-MAX-VISIT(V_s), we may possibly add POIs belonging to V and V_s to obtain a feasible solution to TTDP-M(V). This valid inequality is proven experimentally to be effective because the value of TTDP-MAX-VISIT(V_s) is much less than $|V_s| - 1$.

Note that the master formulation is a weak relaxation of the original formulation (no time-window constraints), so the number of POIs in a candidate solution s may be much larger than the number of POIs in the feasible solutions. In this case, solving TTDP-MAX-VISIT(V_s) is time consuming, and TTDP-MAX-VISIT(V_s) and V_s can be reduced to make constraint (33) more effective. Algorithm 3 identifies a small subset of V_s that violates the time-window constraints and provides another opportunity to find new feasible solutions. Algorithm 3 orders the POIs in s by placing mandatory POIs at the beginning and then sorting the remaining POIs decreasingly by their score. Scores are sorted in decreasing order because this approach matches our branching strategy. Next, we select a subset of POIs such that the total score is better than

the current best solution. If we cannot visit all POIs of this subset, we add a cut to the master model and stop the procedure. Otherwise, we seek improved solutions by calling the local branching and variable neighborhood search procedure and then continue our steps until a violated subset is found. Algorithm 3 terminates because we know that the input of the algorithm is a solution s in which TTDP-S(V_s) is infeasible.

Algorithm 3: Determination of violated sets

Input: A set of POIs $\{u_1, u_2, \dots, u_m\}$ which constitutes an infeasible solution s to TTDP-M(V)

Output: A subset of POIs that violates time-window constraints

- 1 Order the POIs in s such that
- 2 - Mandatory POIs are placed at the beginning in the resulting list
- 3 - Other POIs are sorted decreasingly by score
- 4 Determine the smallest value of k so that $\sum_{1 \leq i \leq k} p_{u_i} \geq v(best)$
- 5 Solve TTDP-MAX-VISIT($\{u'_1, \dots, u'_k\}$)
- 6 **if** TTDP-MAX-VISIT($\{u'_1, \dots, u'_k\}$) $\leq k - 1$ **then**
- 7 Add the corresponding cut (33) to TTDP-M(V) considering that $V_s = \{u'_1, \dots, u'_k\}$
- 8 **else**
- 9 Call LB + VNS($\{u'_1, \dots, u'_k\}$) to find improved solutions
- 10 Let $k \leftarrow k + 1$ and return to Step 4 of this algorithm

Processing an internal node.

When n is an internal node (i.e., the second case, lines 14–27), we want to determine whether branching on unfixed variables of n is interesting. We start by identifying POIs that are assumed to be visited (line 14, denoted $\bar{s}$). These POIs actually correspond to the variables y_i , which are fixed to be 1 in the current node n . First, if no feasible initial solution is found, a call to LB + VNS($\bar{s}$) is made to obtain, as soon as possible, an initial solution (line 16). The main idea of lines 17–27 is to prune the search tree as soon as possible. When it is impossible to visit all POIs in $\bar{s}$ (lines 20–22), we add a cut (33) to the master problem and stop branching on this node. Otherwise, if all POIs in $\bar{s}$ can be visited, we call LB + VNS($\bar{s}$) to find improved solutions only if $\bar{s}$ is at least as large as the current best solution. Since all these steps (lines 20–27) can be time consuming, we check the feasibility of the sub-problem only when the total score of POIs that are planned to visit is at least as large as the current best $[v(\bar{s}) > v(best)]$ or when the number of POIs in $\bar{s}$ is sufficient ($|\bar{s}| > l_{feasible}$). Candidate solutions also need to satisfy lower-bound category constraints. The quantity $l_{feasible}$ estimates an upper bound for the number of POIs in any feasible trip, and this value is updated dynamically as follows:

- Initially, $\alpha = 0$ and $\beta = 0$, where α , β denote the total of the length of feasible trips and the number of trips that we have checked, respectively.
- After each call to TTDP-MAX-VISIT(V_s) (or when the best solution $best$ is updated):
 - $\alpha \leftarrow \alpha + \text{TTDP-MAX-VISIT}(V_s)$ (or $+|V_{best}|$),
 - $\beta \leftarrow \beta + 1$.
- $l_{feasible} \leftarrow \lceil \frac{\alpha}{\beta} \rceil$.

Although TTDP-MAX-VISIT(V_s) can be solved efficiently in most cases (e.g., from milliseconds up to a few seconds during the experimentation), it does not need to be solved optimally. If the resolution time exceeds 15 s, we replace TTDP-MAX-VISIT(V_s) by its upper bound, which is $\min(|V_s| - 1, \lceil \text{UB}(\text{TTDP-MAX-VISIT}(V_s)) \rceil)$, where $\text{UB}(\text{TTDP-MAX-VISIT}(V_s))$ returns an upper bound while solving TTDP-MAX-VISIT(V_s).

While exploring the search tree, many options may be available from which to select the node $n \in \aleph$ to be processed. In the proposed algorithm, we process node n with the largest relaxation value, and

we branch on an unfixed variable that is active in the current node (e.g., $y_i > 0$) and that has the largest score p_i . Experimentation shows that such a strategy is effective and helps to reduce the upper bound more than other strategies.

Finally, the E-B&S variant is obtained by replacing the condition “TTDP-S(V_s) is feasible” in line 7 by “ $v(\text{TTDP-R}(V_s)) = v(s)$ ” because we know that, if TTDP-S(V_s) is feasible, then $\text{TTDP-R}(V_s) = v(s)$. We also add constraint (31) to TTDP-R(V_s) before solving this problem because we are only interested in solutions that are better than the current best solution while solving the restricted problem. This valid inequality also helps to accelerate the resolution of TTDP-R(V_s) because it tightens the search space of TTDP-R(V_s).

In the following sections, we discuss in more details the preprocessing, valid inequalities, and local branching and variable neighborhood search components of our algorithm.

4.2.1. Preprocessing

As mentioned previously, preprocessing should be done to tighten the master and the sub-problem formulation. In our problem, a series of steps is considered, including (1) TSPTW-like preprocessing to eliminate infeasible arcs and tighten time windows [see, e.g., Dumas et al. (1995)], (2) deduction from the implication set, and (3) deduction from the exclusion set.

If $x \rightarrow y$ is an implication, the following rules are used to refine the implication set:

- If x is a mandatory POI, then y may be considered a mandatory POI.
- If y is a mandatory POI, we remove this rule from the implication set.

After applying these rules, the implication set includes only rules where all associated POIs are optional POIs.

Let $\text{imp}(x) \subseteq V$ be the set of POIs that the tourist visits when he/she visits POI x . For example, if the implication set includes rules $\{x \rightarrow y, x \rightarrow z, y \rightarrow r\}$, then $\text{imp}(x) = \{x, y, z, r\}$. Actually, $\text{imp}(x)$ can be built by using a traversal method in graph theory. The following observations are used to refine the exclusion set E :

- If $(u, v) \in E$ and $u, v \in M$, then E is an invalid instance.
- If $(u, v) \in E$ and $u \in M$, we remove v from the set of POIs.
- If the visit of all POIs in $\text{imp}(x) \cup \text{imp}(y)$ violates budget or category constraints, then (x, y) is an exclusion pair.
- If $\exists u \in \text{imp}(x)$ and $\exists v \in \text{imp}(y)$ where $(u, v) \in E$, then (x, y) is also an exclusion pair.

4.2.2. Valid inequalities to strengthen master and sub-problem

We exploit time-window constraints and duration constraints to establish several valid inequalities to tighten the master problem. First, if we cannot visit both POIs i and j in the same trip because the time-window constraint is violated (i.e., $e_i + \tau_{ij} + t_i > l_j$ and $e_j + \tau_{ji} + t_j > l_i$), we have

$$y_i + y_j \leq 1 \quad \forall i, j \in V, \quad e_i + \tau_{ij} + t_i > l_j, \quad e_j + \tau_{ji} + t_j > l_i. \quad (34)$$

Next, we develop valid inequalities related to the duration constraint. If we cannot visit both POIs i and j because of the duration constraint, $l_i + D < e_j$ or $l_j + D < e_i$, then we have

$$y_i + y_j \leq 1 \quad \forall i, j \in V, \quad \min(l_i, l_j) + D < \max(e_i, e_j). \quad (35)$$

The duration constraint also ensures that the total travel time and visit duration must be less than D . Therefore, we have

$$\sum_{i \in V \setminus N} (\tau_i^{\text{out}} + t_i) y_i \leq D, \quad (36)$$

$$\sum_{i \in V \setminus 0} (\tau_i^{\text{in}} + t_i) y_i \leq D, \quad (37)$$

where $\tau_i^{\text{out}} = \min_{(i,j) \in A} \tau_{ij}$ and $\tau_i^{\text{in}} = \min_{(j,i) \in A} \tau_{ji}$ denote the minimum outbound travel time and minimum inbound travel time for a given POI i , respectively.

To tighten the sub-problem formulation TTDP-S(V_s), we rely on another relaxation of the duration constraint. These constraints state that the total travel time and visiting time should not exceed the duration of the visit:

$$\sum_{(i,j) \in A} x_{ij}(\tau_{ij} + t_i) \leq D, \quad (38)$$

$$\sum_{(i,j) \in A} x_{ij}(\tau_{ij} + t_j) \leq D. \quad (39)$$

We exploit the valid inequality (33) by solving exactly a variant of TTDP-MAX-VISIT(V) but with all TTDP constraints in a maximum of 60 s and adding a cut of the form $\sum_{i \in V} y_i \leq \text{TTDP-MAX-VISIT}(V)$ to TTDP-M(V) to limit the maximum number of POIs in any candidate solution of the master problem. This approach also helps tighten the search space of the master problem and improve overall performance.

4.2.3. Local branching and variable neighborhood search

The LB+VNS(V_s) procedure is based on the ideas of local branching (Fischetti and Lodi, 2003) and variable neighborhood search (Mladenovic and Hansen, 1997), which are two well-known heuristic approaches to search for new, improved solutions. Given a solution s and a parameter d , we define a neighborhood $N_s = \{v \in V \mid \exists u \in V_s \text{ such that } \tau_{uv} \leq d\}$ of s , which consists of POIs in V that either belong to s or are close to at least one POI in s . Here, $\tau_{uv} \leq d$ indicates that the distance between u and v does not exceed d . Since N_s consists of POIs close to V_s , it is possible to insert some POIs from $N_s \setminus V_s$ into the current solution or swap POIs between $N_s \setminus V_s$ and V_s to obtain new solutions. We solve TTDP-LB(N_s, k, d), which is the model of TTDP(N_s) plus the local branching constraint (40). This constraint forces the total number of POIs that we can either remove from s and/or insert into s to be at most k :

$$\sum_{i \in V_s} (1 - y_i) + \sum_{i \in N_s \setminus V_s} y_i \leq k. \quad (40)$$

The algorithm maintains a data structure named m in which $m(s)$ gives how many times TTDP-LB(N_s, k, d) has been solved for a particular solution s . If TTDP-LB(N_s, k, d) has yet to be solved for any pair (k, d) , $m(s)$ returns 0.

Algorithm 4: Local Branching and Variable Neighborhood Search: LB + VNS(s)

Input: A (feasible) solution s to TTDP(V)
Output: A (possible) new, improved solution

```

1  $m(s) \leftarrow m(s) + 1$ ;
2 if  $m(s) > |\Omega|$ , stop;
3  $(k, d) \leftarrow$  the  $m(s)$ th  $(k, d)$  pair in  $\Omega$ —the set of all  $(k, d)$  pairs;
4 while  $m(s) \leq |\Omega|$  do
5   Define the neighborhood  $N_s$  of  $s$  based on the value  $d$ ;
6   Solve TTDP-LB( $N_s, k, d$ ) with the cut (40) for the parameter  $k$ ;
7   if a new, improved solution  $s'$  is found then
8     Update  $s \leftarrow s'$ ;
9      $m(s) \leftarrow 1$ ;
10     $(k, d) \leftarrow$  the 1st  $(k, d)$  pair in  $\Omega$ ;
11  else
12     $m(s) \leftarrow m(s) + 1$ ;
13     $(k, d) \leftarrow$  the  $m(s)$ th  $(k, d)$  pair in  $\Omega$ ;
14 return  $s$ ;
```

In our implementation detailed in Algorithm 4, we maintain a list of (k, d) pairs in an ordered list Ω , and each element in Ω is used at most once. For each input s , we determine the number of times we call TTDP-LB(N_s, k, d) for some pairs of (k, d) (line 1). This can be done by using a

Table 2

Characteristics of original benchmark instances (Souffriau et al., 2013).

	#Instances	#POIs	#Categories
Solomon et al.	29	100	10
Cordeau et al.	8	48–233	10

hashing technique. We hash the POIs in s into an integer number, and we store the number of times TTDP-LB(N_s, k, d) has been called for each s . If all pairs in Ω are used, we stop (line 2). Otherwise, we define the neighborhood N_s and solve the corresponding TTDP-LB(N_s, k, d). If a new, improved solution s' is found, we reset (k, d) to the first pair in Ω . Otherwise, we adjust (k, d) to the next pair in Ω .

It is worth noting that although B-B&C, B-B&S, E-B&C, and E-B&S do not necessarily explore the same search tree for the original TTDP, given enough time, they all converge to an equivalent (in terms of the objective function) optimal solution.

5. Experiments

This section assesses the performance of the proposed algorithms. The algorithms were implemented in C++. Experiments were run on workstations of CalculCanada's servers¹ in which we allow 50 Gigabyte of memory for CPLEX. We used CPLEX Concert 12.8 API to formulate and solve all MIP formulations. We relied on CPLEX's callback library to access integer solutions and internal nodes.

When testing the proposed approaches, all CPLEX parameters were set to their default values except for (i) the number of threads, *Threads*, which is set to 1 and (ii) the heuristic strategy, *HeuristicFreq*, which is disabled when solving TTDP-M(V). We let the algorithms run until they found an optimal solution or one hour of CPU time elapsed. When using CPLEX as a standalone solver, all parameters were set to their default values. In this case, we let CPLEX run for 1 h or 5 h, depending on the experiment.

In our case, since the profits p_i are integers, the sum $\sum_{i \in V} y_i p_i$ is also an integer. Therefore, optimality is reached when the difference between the upper and lower bounds is less than 1. We implemented the checking steps, valid inequalities (31)–(33), node selection, and branching strategy as callbacks. When we do not activate the proposed node selection and own branching strategy (i.e., in our B-B&C, B-B&S, E-B&C and E-B&S), we let CPLEX decide which nodes to process. Regarding the local branching and variable neighborhood search procedures, Ω includes three pairs in our implementation, $\{(10, 15), (15, 10), (20, 5)\}$. We solve TTDP-LB(N_s, k, d) for at most 60 s for each pair.

We tested our algorithms on instances adapted from the MCTOPMTW (called SC-O; see Table 2), proposed by Souffriau et al. (2013). Briefly, the original set consists of 37 instances, of which 29 are extended from the well-known Solomon dataset (Solomon, 1987), and the remaining 8 extended from instances proposed by Cordeau et al. (1997). The initial and destination locations are assumed to be the same. The duration constraint is assumed to be the duration of the time window at the departure location. To model the budget constraint, the authors gave to each POI a random entrance fee between 0 and 99. To model the category constraint, they gave each POI i a 0.2 probability of belonging to category c for each $c \in C$ and randomly assigned POIs to categories. Therefore, each POI may belong to several or zero categories. Each category has an average of 22 POIs and 12 POIs belong to no category.

We generated 3 sets of instances based on the Souffriau et al. (2013) benchmarks described above. We now explain how we adapted their set to our TTDP. In our instances, we use the same upper-bound and budget constraints than in the original set. Since Souffriau et al. (2013) consider multiple time windows whereas our problem assumes a single

¹ https://docs.computeCanada.ca/wiki/Compute_Canada_Documentation

Table 3

Notation.

Legend	Meaning
#Optimal	Number of instances that are optimally solved
#Fea. Sol.	Number of instances where at least one feasible solution is found
#Reject	Number of nodes pruned while branching
#Check	Number of sub or restricted problems solved
LB	Lower bound — which is the value of the best feasible solution
UB	Upper bound — which is the upper bound of any feasible solution
Unsolved gap (%)	Gap between LB and UB, $100\% \times (UB - LB)/UB$, of unsolved instances
B-B&C or E-B&C	Baseline or enhanced branch-and-check algorithms
B-B&S or E-B&S	Baseline or enhanced branch-and-solve algorithms

time window, the time window for each POI in our set is defined by the time interval [opening of the first time window, closing of the last time window] of the corresponding POI in the original set. To generate the additional constraints, we used the following rules. Let $s = (u_0, u_1, u_2, \dots, u_m)$ and its associated POI set $V_s = \{u_0, u_1, u_2, \dots, u_m\}$ be a solution to the MCTOPMTW obtained after solving the problem by applying CPLEX for 10 h. The idea is to add the new constraints while ensuring that s remains a feasible solution to the resulting TTDP instance. First, the mandatory POIs are randomly selected from V_s . Depending on the instance set, the number of mandatory POIs varies between 2 and 5 (sets 1 and 2) and minimum of either $\frac{2}{3}|V_s|$ or 10 POIs (sets 3). While the choice of these intervals is totally arbitrary, the intuition is to try to find a good balance between instance difficulty and realism. Indeed, one can expect the difficulty of the instance to decrease with the number of mandatory POIs, and, in practice, even highly touristic regions only have a handful of attractions that are must-sees to every single tourist. Second, to generate lower-bound constraints, we first compute nb_c , which is the number of POIs of type c that is in s . Next, lb_c is set to a random number in the interval $[0, nb_c]$. Last, but not least, to generate precedence constraints, implication constraints, and exclusion constraints, we enumerate all possible constraints and select a portion of them. Given a random ordered pair (p, q) of $V \times V$ POIs, the constraints are enumerated as follows:

- Implication constraint: (p, q) can be in I if either (1) $p, q \in V_s$, or (2) $p \in V \setminus V_s, q \in V$.
- Exclusion constraint: (p, q) can be in E if at least p or q is in $V \setminus V_s$.
- Precedence constraint: (p, q) can be in P if (1) $p = u_i \in V_s, q = u_j \in V_s$ with $i < j$, or (2) $p \in V \setminus V_s, q \in V$.

These enumeration strategies ensure that s remains a feasible solution to the corresponding new TTDP instance. Our three instance sets have the following characteristics:

- Set 1 has 37 instances. It corresponds to the 1-team set in the Souffriau et al. (2013) paper. Each instance has an number of mandatory POIs randomly selected in the interval 2 to 5. To set the precedence, implication, and exclusion constraints, we randomly selected 1% of all the enumerated constraints of each type.
- Set 2 also consists of 37 instances. It correspond to the 2-team set in the Souffriau et al. (2013) paper. Similar to Set 1, here each instance has a number of mandatory POI randomly selected in the interval 2 to 5. To set the precedence, implication, and exclusion constraints, we randomly selected 2% of all the enumerated constraints of each type. It is worth noting, that budget and the upper bound values are roughly doubled with respect to those of Set 1.
- The third set include 74 instances obtaining by extending the sets 1 and 2. Each instance in this set, has the same values of upper-bound and budget constraints than the corresponding instance in Set 1 and Set 2. However, instances in this set have more mandatory POIs. More specifically, given a solution s to the original instance (e.g. MCTOPMTW instances), the number of mandatory POIs is set to the minimum value between $\frac{2}{3}|V_s|$ and 10. To set the precedence, implication, and exclusion constraints, we randomly selected 2% of all the enumerated constraints of each type.

Table 4

The results obtained by CPLEX and the proposed algorithms for three datasets of TTDP.

		CPLEX				E-B&C	E-B&S
		1 CPU		8 CPUs			
		1 H	5 H	1 H	5 H		
Set 1	#Fea. Sols	31	34	34	34	37	37
	#Optimal	7	11	12	13	29	28
	Unsolved gap	47.4	39.7	33.7	33.6	24.5	26.9
Set 2	#Fea. Sols	36	36	36	37	37	37
	#Optimal	8	9	9	11	24	23
	Unsolved gap	47.6	44.5	34.1	30.3	14.5	17.5
Set 3	#Fea. Sols	61	68	67	69	74	74
	#Optimal	31	35	42	45	70	70
	Unsolved gap	50.7	43.5	44.7	46.5	10.0	17.4

Table 5

Notation for benchmark instances.

Legend	Constraints
SC-O	Original constraints
SC-M	Mandatory constraints
SC-LB	Lower-bound category constraints
SC-I	Implication constraints
SC-C	Exclusion constraints
SC-P	Precedence constraints

Our first experiment aims to assess the overall value of the proposed approaches with respect to black-box optimization. Therefore, we report and compare results obtained by our best approaches, E-B&C and E-B&S, and those obtained by CPLEX running with different configurations. In the second experiment, we compare the baseline versions of the methods (B-B&C and B-B&S) to their enhanced versions (E-B&C and E-B&S) to measure the value of going the extra mile with respect to the classical B&C framework. In the third and last experiment, we report and analyze the contribution of each algorithmic component to the performance of E-B&C and E-B&S.

Before going into details, Table 3 explains the notation used in the following tables. We report the number of instances for which optimal solutions are found (#Optimal), the number of instances where feasible solutions are found (#Fea. Sol.), the number of times candidate solutions s are checked (#Check), and the number of times an internal node is pruned while branching (#Reject).

5.1. Experiment 1: overall performance

Table 4 reports the results obtained by CPLEX solving the model proposed in Section 3 enhanced with all the pre-processing techniques introduced in Section 4.2.1. We report on 4 possible CPLEX configurations built by combining the number of CPUs available to the solver and a CPU time limit. For instance, under configuration 8 CPUs - 5H, CPLEX ran in parallel on 8 CPUs (i.e., up to 32 threads) during a time of 5 h. The table also reports the results delivered by E-B&C and E-B&S running on 1 thread during 1 h. Row “Unsolved gap” reports an average across the each instance set.

Table 6

Results of baseline branch-and-check and branch-and-solve approaches.

		SC-O	SC-M	SC-MLB	SC-MLBI	SC-MLBIC	SC-MLBICP
B-B&C	#Optimal	0	5	6	12	12	12
	#Fea. Sol.	7	9	9	15	15	14
	Unsolved gap (%)	86.32	78.02	77.30	61.37	61.42	62.92
	#Check	230,519	186,036	192,064	142,740	139,977	138,538
B-B&S	#Optimal	0	2	4	10	11	11
	#Fea.Sol	37	37	37	37	37	37
	Unsolved gap (%)	39.75	35.95	35.55	26.91	25.91	25.76
	#Check	19,893	17,877	21,385	18,499	15,337	16,598
CPLEX	#Optimal	8	9	10	11	12	13
	#Fea.Sol	37	37	33	35	34	34
	Unsolved gap (%)	35.07	32.44	41.77	34.46	33.65	33.60

Table 7

Percent difference between upper-bound values returned by CPLEX 5h and those return by the baseline branch-and-check (B-B&C) and branch-and-solve (B-B&S) approaches.

	SC-O	SC-M	SC-MLB	SC-MLBI	SC-MLBIC	SC-MLBICP
$\frac{UB_{B-B&C} - UB_{CPLEX}}{UB_{B-B&C}}$ (%)	5.59	4.09	3.33	-1.33	1.19	1.64
$\frac{UB_{B-B&S} - UB_{CPLEX}}{UB_{B-B&S}}$ (%)	7.36	6.60	5.73	2.22	4.91	5.09

The results show that E-B&C and E-B&S consistently outperform CPLEX in the three instance sets. First, both of our approaches deliver feasible solutions in all instances, which is not the case for CPLEX, even running on its high-computational-resource configuration. Second, our methods returned optimal solutions on nearly twice the number of instances than CPLEX running on its best configuration. Finally, on instances without a probable optimal solution E-B&C and E-B&S report optimality gaps that are nearly half of those reported by CPLEX. It is worth noting, that E-B&C and E-B&S deliver these results with only a fraction of the computational resources given to CPLEX and employing execution times that remain reasonable for an exact method.

We conducted a second analysis aiming to better understand the impact of the different constraints on the difficulty of the problem. More specifically, we solved each of the 148 instances taking into account 6 different sub-sets constraints using CPLEX, E-B&C and E-B&S, and compared the number of optimal solutions found by each approach. Table 5 introduces the naming conventions for each of the 6 subsets of constraints. SC-O refers to instances with original constraints: budget constraints, duration constraints, and upper bound category constraints. SC-M means that mandatory constraints are considered (or $M \neq \emptyset$) in addition to those considered in SC-O, SC-MLB means that mandatory constraints and lower-bound category constraints are considered, etc. Instances called SC-MLBICP are therefore instances of the TTDP with all constraints applied. Given that Set 1 and Set 2 have 37 instances; and Set 3 has 74 instances; the experiment consisted in solving a total of 888 instances (222 for Set 1 and Set 2; and 444 for Set 3).

Fig. 1 summarizes the results. The data shows that instances with only the “original” constraints are considerably harder to solve than instances with “all” the constraints. The reason is simply that their solution space is much larger and more difficult to explore for our algorithms. Indeed, since there are more feasible solutions, the algorithms benefit less from the feasibility cuts generated during the exploration. A quick look at the results obtained for Set 2 and 3 confirms this intuition. For instance, let us focus on the performance of E-B&C. On Set 1, this approach delivered optimal solutions for 65.31% of the instances (145/222) but this figure is 74.77% (332/444) on Set 3. Recall that instances in Set 3 have a larger number of mandatory POIs and a larger number of implication, exclusion, and precedence constraints than those in sets 1 and 2. Therefore, instances in Set 3 are “more constrained” and better suited for our approaches. The opposite can be observed on Set 2, in which instances have looser upper bound, duration, and budget constraints.

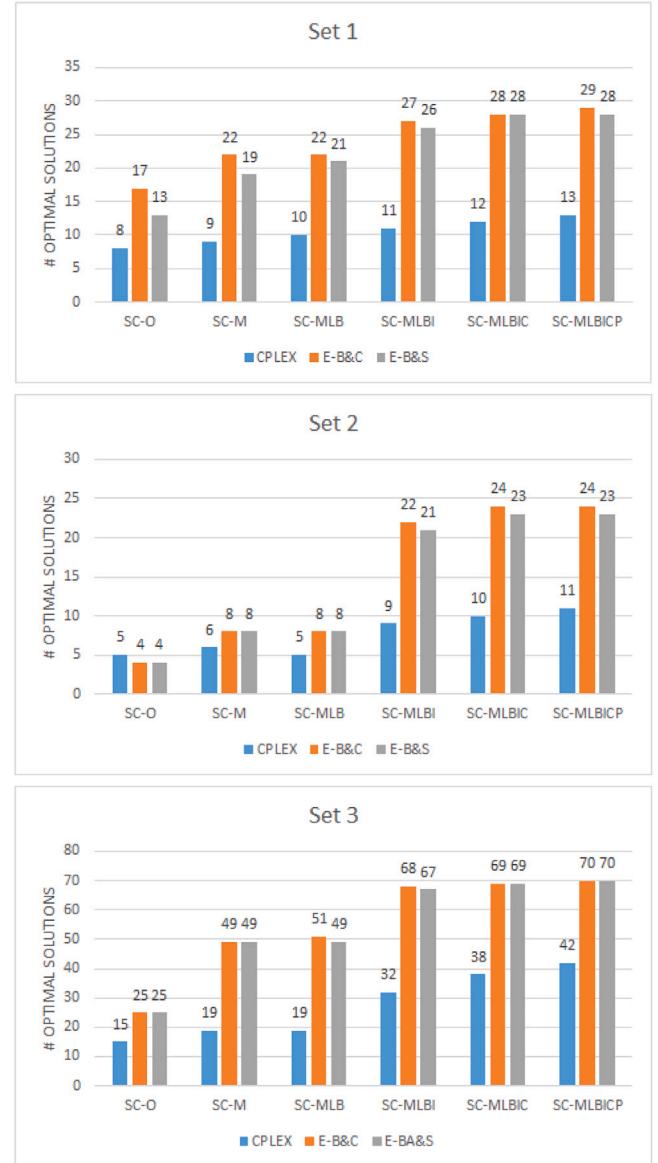


Fig. 1. The number of solved instances by CPLEX 5H-8 CPUs, E-B&C and E-B&S 1H for each configuration.

5.2. Experiment 2: B-B&C vs. B-B&S and E-B&C vs. E-B&S

To assess the benefits of the “solve” component and the enhancements to the baseline algorithms, we ran our four approaches on each

Table 8

Performance of the enhanced branch-and-check and branch-and-solve approaches.

		SC-O	SC-M	SC-MLB	SC-MLBI	SC-MLBIC	SC-MLBICP
E-B&C	#Optimal	17	22	27	28	29	29
	#Fea. Sol.	37	37	37	37	37	37
	Unsolved gap (%)	21.95	24.04	26.63	24.81	22.07	24.53
	#Check	368	281	386	425	407	395
	#Reject	6,993	3,785	2,307	2,261	1,999	1,542
	$\frac{LB_{E-B&C} - LB_{CPLEX}}{LB_{E-B&C}}$ %	3.06	3.53	10.6	5.7	8.5	7.5
E-B&S	$\frac{UB_{E-B&C} - UB_{CPLEX}}{UB_{E-B&C}}$ %	-21.50	-23.63	-21.49	-21.95	-18.45	-18.21
	#Optimal	13	19	21	26	28	28
	#Fea. Sol.	37	37	37	37	37	37
	Unsolved gap (%)	20.54	22.79	25.69	26.16	26.48	26.99
	#Check	272	174	291	190	240	218
	#Reject	7,923	3,843	1,615	965	863	830
	$\frac{LB_{E-B&S} - LB_{CPLEX}}{LB_{E-B&S}}$ %	3.31	3.35	10.8	5.6	8.2	7.4
	$\frac{UB_{E-B&S} - UB_{CPLEX}}{UB_{E-B&S}}$ %	-19.36	-21.69	-20.40	-20.42	-16.85	-16.18

of the 6 instance configurations for Set 1 (a total of 222 instances). Table 6 reports the results obtained by the baseline branch-and-check and branch-and-solve approaches (Algorithm 4.2.1, B-B&C and B-B&S), where the valid inequalities (38) and (39) have been added to the sub-problem formulations TTDP-S(V_s) and TTDP-R(V_s), respectively. Table 6 presents the results. For reference, we added to the bottom of the table the results delivered by CPLEX running on its best configuration (i.e., 8 CPUs - 5 h).

As the data shows, the results obtained by the baseline approaches are not better than those returned by CPLEX. Note also that, although numerous candidate solutions are checked, B-B&C fails to find feasible solutions to many of instances. One plausible explanation for this result is that the master formulation is a weak relaxation of the original problem, so the candidate solutions usually violate time-related constraints. On the other hand, B-B&S is able to find feasible solutions for all the 37 instances in all the 6 configurations. Similarly, B-B&S delivers solutions with smaller optimality gaps than B-B&C. Note, however, that these two advantages do not necessarily translate into a larger number of optimal solutions; the data suggests that the performance of B-B&S in that metric is comparable to that of B-B&C and CPLEX. One possible explanation is that B-B&S employs more time than B-B&C to solve the sub-problem. As a result, B-B&S checks fewer candidates (see lines #Check for both approaches) and has an inferior upper bound. The latter is shown in Table 7 where the final upper bounds found by B-B&C and B-B&S are compared to those delivered by CPLEX.

Table 8 shows the results of the enhanced branch-and-check and branch-and-solve approaches (Algorithm 2, E-B&C and E-B&S) running on a single thread with a time limit of 1 h. When the problem is more constrained, the number of solved instances increases, as seen in the row “#Optimal.” This result is attributed to the solution space of the master problem being much smaller when the problem is more constrained. We solve 29 (or 28) of the 37 instances of the SC-MLBICP benchmark set (which is our TTDP) with E-B&C (or E-B&S), whereas we solve only 17 (or 13) of the 37 instances of the original set SC-O with E-B&C (or E-B&S). The more constraints a problem has, the more effective the approach is because the search space of the master problem is much more strongly bounded, resulting in significantly tighter upper bounds and fewer candidate solutions to check.

Also, the values of “#Check” and “#Reject” show that much fewer TTDP-S(V_s) or TTDP-R(V_s) need to be solved than for the baseline branch-and-check and branch-and-solve, which indicates the strength of the proposed cuts and branching strategy in terms of eliminating invalid candidate solutions and invalid partial solutions. The optimal gap of unsolved instances remains large mainly because of instances of the Cordeau class.

The rows “LB vs LB CPLEX 5h” and “UB vs UB CPLEX 5h” show the improvement in our lower and upper bounds compared with those obtained by CPLEX 5h. The percent improvement is calculated as by

Table 9

Contribution of valid inequalities (38) and (39) to strengthening the upper bound.

		SC-M	SC-MLB	SC-MLBI	SC-MLBIC
Without (38) & (39)	#Fea. Sol.	9	7	13	13
	#Check	76 752	76 959	69 303	69 416
With (38) & (39)	#Fea. Sol.	9	9	15	15
	#Check	186 036	192 064	142 740	139 971
$\frac{UB_{With} - UB_{Without}}{UB_{With}}$ (%)		-0.98	-1.22	-1.04	-1.11

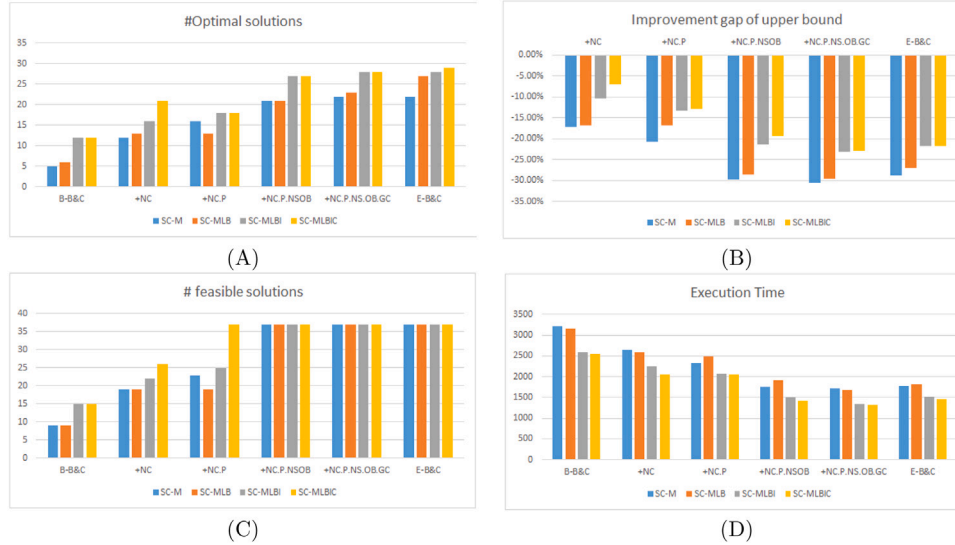
$100\% \times (LB_{E-B&C} - LB_{CPLEX 5h}) / LB_{E-B&C}$ and $100\% \times (UB_{E-B&C} - UB_{CPLEX 5h}) / UB_{E-B&C}$, for the E-B&C, and similarly for E-B&S. We note that while the E-B&C and E-B&S find feasible solutions to all instances and CPLEX cannot, CPLEX produce averagely better primal solutions for a subset of unsolved instances of both methods, mainly those of Cordeau instances. Statistically, CPLEX finds better solutions 36 over 222 instances. Vice versa, E-B&C finds better solutions for 78 over 222 instances. However, it indicates that there is improvement gap for better primal heuristics. Next, row “UB vs UB CPLEX 5h” explains the contributions of proposed cuts and branching strategy by large negative improvement gap of the upper bound values. Combining with lower bound values found by local branching heuristics, the proposed approach is able to find optimal solutions and proves the optimality in many more instances than CPLEX.

Finally, note that E-B&C performs slightly better than E-B&S, which we attribute to the longer time required to solve TTDP-R(V_s) than to solve TTDP-S(V_s), so fewer candidate solutions and nodes are explored.

5.3. Experiment 3: assessing the value of proposed components

This section examines the contribution of the proposed cuts and algorithmic components. All results were obtained by using the baseline branch-and-check algorithm and additional components. Table 9 details the contributions of the cuts (38) and (39). The evaluation was done by using the baseline branch-and-check algorithm since it gives a clearer view of how the cuts work. Recall that TTDP-S(V_s) is actually a variant of the traveling salesman problem with time windows (TSPTW), which is an NP-hard problem even in the sense of finding feasible solutions Savelsbergh (1985). We obtain candidate solutions with checking steps that are very costly in terms of execution time when cuts (38) and (39) are not used. Applying these two cuts significantly accelerates the checks, resulting in more candidate solutions to be checked and thereby improving the performance of the proposed algorithm. This approach also raises the question of how to efficiently solve small-size traveling salesman problems with time windows (e.g., constraint programming could be a useful approach).

Table 10
Summary of contributions of each algorithmic component.



Next, Table 10 examines the contributions of each algorithmic component proposed. The baseline branch-and-check algorithm (Algorithm 1) is denoted “B-B&C”. The cut (33), preprocessing (Section 4.2.1), node selection and branching strategy (processing an internal node in Section 4.2), greedy checking (Algorithm 3), and the local branching and variable neighborhood search (Section 4.2.3) are denoted “NC,” “P,” “NSOB,” “GC,” and “LB,” respectively. The configuration “+NC” thus means that we implement the baseline branch-and-check plus the new cut component, the configuration “+NC.P” means that we implement the baseline branch-and-check plus the new cut component and the preprocessing components, etc. The configuration “+NC.P.NSOB.GC.LB” corresponds then to our enhanced branch-and-check and is denoted “E-B&C.”

Table 10A describes how each algorithmic component contributes to the number of solved instances. The four left-most columns present the number of proven optimal solutions delivered by B-B&C for each set. Each of the next four columns presents the number of optimal solutions returned by a configuration consisting of the B-B&C and new algorithmic components. Table 10B compares the upper bound obtained by using the new algorithmic components with those obtained by using the baseline approach. The numbers show the important contribution of each algorithmic idea in terms of improving the upper bounds, especially valid inequalities, preprocessing, and the branching-and node-selection strategy. Note that the average UB gaps returned by E-B&C (4 last columns) are slightly inferior to those returned by +NC.P.NSOB.GC. The most likely explanation is that the CPU time devoted to the additional LB component leads to a smaller number of candidate to check and to cut, which translates into lower UBs on instances that are not solved to optimality. Table 10C shows that the feasibility issues exhibited by the baseline approach can be solved either by a local branching and variable neighborhood search or by a node selection and branching strategy NSOB (in the configuration +NC.P.NSOB and +NC.P.NSOB.GC) where we also check the feasibility for partial solutions. To understand the contribution of local branching and variable neighborhood search procedure, Table 11 shows the improvement of the lower bound value by comparing the E-B&C method with and without the components of (i) greedy checking and (ii) the local branching and variable neighborhood search (first line), and comparing the E-B&C with and without the local branching and variable neighborhood search (second line). We can note that the local branching and variable neighborhood search procedure generates solutions with on average, 3.20% (and 1.35%) improvement for lower bounds compared with solutions generated by the +NC.P.NSOB (and

Table 11

Average improvement of lower bounds when using the local branching and variable neighborhood comparing to lower bounds collected by +NC.P.NSOB and +NS.P.NSOB.GC components.

	SC-M	SC-MLB	SC-MLBI	SC-MLBIC
$\frac{LB_{E-B\&C} - LB_{NC.P.NSOB}}{LB_{E-B\&C}}$ (%)	2.77	3.09	3.53	3.43
$\frac{LB_{E-B\&C} - LB_{NC.P.NSOB.GC}}{LB_{E-B\&C}}$ (%)	1.08	1.16	1.62	1.53

+NC.P.NSOB.GC) configuration for those unsolved instances. Also, this approach helps reducing overall execution time because finding good solutions earlier helps excluding poor candidates from consideration. Finally, Table 10D reports the average execution time in seconds for all instances on each set whether they were solved to optimality or not. The results show that the average running time is reduced when a new component is added and that all algorithmic components contribute to improve the performance of the baseline branch-and-check approach. It is worth noting that the average CPU times reported by E-B&C are slightly longer than those reported by +NC.P.NSOB.GC. The difference can again be attributed to the LB component which is called on every improving solution. This small increase on execution time is largely offset by the better LBs leading to a larger number of optimal solutions.

6. Conclusion

As part of the SmartLoire Project, this paper studies an orienteering problem called the tourist-trip design problem with rich constraints. The TTDP consists in delivering a route plan to tourists according to their preferences, budget, time constraints, and tourism constraints. Unlike most studies in this field, we consider not only the most common characteristics such as duration, budget, and category constraint but also practical constraints, including implication rules, precedence rules, and exclusion rules.

We propose a set of four exact methods based on the branch-and-check approach to solve the problem. The experiment with the total 148 instances of different characteristics shows effectiveness of the proposed approaches and their customized algorithmic components. Overall, our methods found optimal solutions to 123 out of the 148 instances; which is significantly better than the state-of-art solver CPLEX.

A possible direction for future research is to extend the methods to solve the multiple-period tourist trip design problem where, instead of generating a single tour, multiple tours are generated for several days.

We also plan to develop an efficient meta-heuristic approach to power the optimization engine of the SmartLoire platform. The algorithms proposed in this paper can then be used to evaluate the performance of that heuristic approach.

CRedit authorship contribution statement

Duc Minh Vu: Conceptualization, Methodology, Investigation, Software, Writing – original draft, Formal analysis. **Yannick Kergosien:** Conceptualization, Methodology, Resources, Writing – review & editing, Supervision, Funding acquisition, Project administration. **Jorge E. Mendoza:** Conceptualization, Methodology, Resources, Writing – review & editing, Supervision, Funding acquisition, Project administration. **Pierre Desport:** Conceptualization, Methodology, Writing – review & editing.

Acknowledgment

This research was partially funded by the Région Centre - Val de Loire (France) through the project SmartLoire.

References

- Boulakbech, M., Messai, N., Sam, Y., Devogele, T., Etienne, L., 2016. SmartLoire: A web mashup based tool for personalized touristic plans construction. In: 2016 IEEE 25th International Conference on Enabling Technologies: Infrastructure for Collaborative Enterprises (WETICE), pp. 259–260.
- Cordeau, J., Gendreau, M., Laporte, G., 1997. A tabu search heuristic for periodic and multi-depot vehicle routing problems. *Networks* 30 (2), 105–119.
- Dumas, Y., Desrosiers, J., Gélinas, E., Solomon, M.M., 1995. An optimal algorithm for the traveling salesman problem with time windows. *Oper. Res.* 43 (2), 367–371.
- Expósito, A., Mancini, S., Brito, J., Moreno, J.A., 2019. A fuzzy GRASP for the tourist trip design with clustered POIs. *Expert Syst. Appl.* 127, 210–227.
- Feillet, D., Dejax, P., Gendreau, M., 2005. Traveling salesman problems with profits. *Transp. Sci.* 39 (2), 188–205.
- Fischetti, M., Lodi, A., 2003. Local branching. *Math. Program.* 98 (1–3), 23–47.
- Froger, A., Gendreau, M., Mendoza, J.E., Pinson, E., Rousseau, L.-M., 2017. A branch-and-check approach for a wind turbine maintenance scheduling problem. *Comput. Oper. Res.* 88, 117–136.
- Gavalas, D., Konstantopoulos, C., Mastakas, K., Pantziou, G., 2014. A survey on algorithmic approaches for solving tourist trip design problems. *J. Heuristics* 20 (3), 291–328.
- Gavalas, D., Konstantopoulos, C., Mastakas, K., Pantziou, G., Tasoulas, Y., 2013. Cluster-based heuristics for the team orienteering problem with time windows. In: *International Symposium on Experimental Algorithms*. Springer, pp. 390–401.
- Gavalas, D., Konstantopoulos, C., Mastakas, K., Pantziou, G., Vathis, N., 2015. Heuristics for the time dependent team orienteering problem: Application to tourist route planning. *Comput. Oper. Res.* 62, 36–50.
- Gunawan, A., Lau, H.C., Vansteenwegen, P., 2016. Orienteering problem: A survey of recent variants, solution approaches and applications. *European J. Oper. Res.* 255 (2), 315–332.
- Jriji, D., Krichen, S., Madany, F., 2020. A memetic algorithm for the tourist trip design with clustered points of interests. In: 2020 International Multi-Conference on “Organization of Knowledge and Advanced Technologies”(OCTA). IEEE, pp. 1–6.
- Karimi, A., Bashiri, M., 2018. The bi-team orienteering problem with time window. *Int. J. Serv. Oper. Manag.* 30 (2), 203–225.
- Khamsing, N., Chindaprasert, K., Pitakaso, R., Sirirak, W., Theeraviriya, C., 2021. Modified ALNS algorithm for a processing application of family tourist route planning: A case study of buriram in thailand. *Computation* 9 (2), 23.
- Lam, E., Van Hentenryck, P., 2017. Branch-and-check with explanations for the vehicle routing problem with time windows. In: *International Conference on Principles and Practice of Constraint Programming*. Springer, pp. 579–595.
- Lin, S.-W., Vincent, F.Y., 2015. A simulated annealing heuristic for the multiconstraint team orienteering problem with multiple time windows. *Appl. Soft Comput.* 37, 632–642.
- Lin, S., Yu, V.F., 2017. Solving the team orienteering problem with time windows and mandatory visits by multi-start simulated annealing. *Comput. Indust. Eng.* 114, 195–205.
- Lu, Y., Benlic, U., Wu, Q., 2018. A memetic algorithm for the orienteering problem with mandatory visits and exclusionary constraints. *European J. Oper. Res.* 268 (1), 54–69.
- Matl, P., Nolz, P.C., Ritzinger, U., Ruthmair, M., Tricoire, F., 2017. Bi-objective orienteering for personal activity scheduling. *Comput. Oper. Res.* 82, 69–82.
- Mladenovic, N., Hansen, P., 1997. Variable neighborhood search. *Comput. Oper. Res.* 24 (11), 1097–1100.
- Naderi, B., Roshanaei, V., 2020. Branch-relax-and-check: A tractable decomposition method for order acceptance and identical parallel machine scheduling. *European J. Oper. Res.* 286 (3), 811–827.
- Palomo-Martínez, P.J., Salazar-Aguilar, M.A., Laporte, G., Langevin, A., 2017. A hybrid variable neighborhood search for the orienteering problem with mandatory visits and exclusionary constraints. *Comput. Oper. Res.* 78, 408–419.
- Rezki, H., Aghezaf, B., 2018. λ -GRASP With bi-directional path relinking for the bi-objective orienteering problem. *Int. J. Logist. Syst. Manage.* 29 (4), 455–475.
- Roshanaei, V., Booth, K.E., Aleman, D.M., Urbach, D.R., Beck, J.C., 2020. Branch-and-check methods for multi-level operating room planning and scheduling. *Int. J. Prod. Econ.* 220, 107433.
- Sadykov, R., 2008. A branch-and-check algorithm for minimizing the weighted number of late jobs on a single machine with release dates. *European J. Oper. Res.* 189 (3), 1284–1304.
- Savelsbergh, M., 1985. Local search in routing problems with time windows. *Ann. Oper. Res.* 4 (1), 285–305.
- Solomon, M.M., 1987. Algorithms for the vehicle routing and scheduling problems with time window constraints. *Oper. Res.* 35 (2), 254–265.
- Souffriau, W., Vansteenwegen, P., 2010. Tourist trip planning functionalities: State-of-the-art and future. In: Daniel, F., Facca, F.M. (Eds.), *Current Trends in Web Engineering*. Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 474–485.
- Souffriau, W., Vansteenwegen, P., Vanden Berghe, G., Van Oudheusden, D., 2013. The multiconstraint team orienteering problem with multiple time windows. *Transp. Sci.* 47 (1), 53–63.
- Sylejmani, K., Dorn, J., Musliu, N., 2012. A tabu search approach for multi constrained team orienteering problem and its application in touristic trip planning. In: 2012 12th International Conference on Hybrid Intelligent Systems (HIS). IEEE, pp. 300–305.
- Tenemaza, M., Luján-Mora, S., De Antonio, A., Ramirez, J., 2020. Improving itinerary recommendations for tourists through metaheuristic algorithms: an optimization proposal. *IEEE Access* 8, 79003–79023.
- Thorsteinsson, E.S., 2001. Branch-and-check: A hybrid framework integrating mixed integer programming and constraint logic programming. In: *International Conference on Principles and Practice of Constraint Programming*. Springer, pp. 16–30.
- Trachanatzi, D., Rigakis, M., Marinaki, M., Marinakis, Y., 2020. An interactive preference-guided firefly algorithm for personalized tourist itineraries. *Expert Syst. Appl.* 159, 113563.
- Tricoire, F., Romauch, M., Doerner, K.F., Hartl, R.F., 2010. Heuristics for the multi-period orienteering problem with multiple time windows. *Comput. Oper. Res.* 37 (2), 351–367.
- Vansteenwegen, P., Gunawan, A., 2019. *Orienteering Problems: Models and Algorithms for Vehicle Routing Problems with Profits*. Springer Nature.
- Vansteenwegen, P., Souffriau, W., Oudheusden, D.V., 2011. The orienteering problem: A survey. *European J. Oper. Res.* 209 (1), 1–10.
- Vendé, P., Kergosien, Y., Mendoza, J.E., 2021. The charger location problem in hybrid electric bus networks, *Working Paper*.
- Vu, D.M., Hewitt, M., Boland, N., Savelsbergh, M., 2020. Dynamic discretization discovery for solving the time dependent traveling salesman problem with time windows. *Transp. Sci.* 54 (3), 565–583.
- Zhao, Y., Alfandari, L., 2020. Design of diversified package tours for the digital travel industry: A branch-cut-and-price approach. *European J. Oper. Res.* 285 (3), 825–843.